Financial Knowledge and Numerical Tools for Trading Agents: A Factorial Study of Access, Use, and Returns

Anonymous ACL submission

Abstract

Financial agents can be given domain guidance and numerical algorithms, but access does not establish that the agent uses either resource or improves its decisions. We present FIN-SKILLS, a library of financial skills, quantitative interfaces, and executable research checks, and evaluate optional document and algorithm access in a frozen factorial trading study. Qwen2.5-Coder 7B and 14B make 2112 decisions across four conditions, three sampling seeds, and two asset baskets on a shared historical calendar. All conditions see identical adjusted prices and use the same budget, next-close execution, and costs. Full access changes mean cumulative net return relative to the base condition by 5.70 and -0.73 percentage points on the original assets, and 1.69 and 4.70 points on preselected transfer assets, for 7B and 14B respectively. Every full-access group trails equal-weight rebalancing. There are 0 successful skill-document reads and only 31 decisions with a successful algorithm call. Calendar-block intervals span zero for every reported effect. The experiment measures access under limited uptake; it does not establish gains from knowledge consumption or general trading superiority. Separate guard and code-generation audits describe the library's ability to detect specified financial errors, without attributing those effects to the trading intervention. A completed interface-repair follow-up still yields no valid library-arm code deliverables.

1 Introduction

A library can make financial knowledge and quantitative methods available to a trading agent without making the agent consult them. Even when an interface is used successfully, a higher historical return may reflect exposure choices on a particular market path. Evaluating a financial tool library therefore requires observing the chain from access to use to submitted decisions, alongside returns, risk, and simple strategy references.

FIN-SKILLS combines dated financial guidance with numerical interfaces and executable research checks. Domain skills describe suitable libraries and failure modes; method interfaces expose input and parameter contracts; guards check declared assumptions such as timing, corporate actions, and validation splits. These components support different claims. A working algorithm establishes availability, a recorded call establishes use, and a guard result establishes only that its specified check ran.

We ask what changes when a trading agent receives skill-document access, numerical-tool access, or both. The main experiment separates these two access interventions in a four-cell design with identical price data. It retains every planned decision, including failed calls and missing submissions. An original basket is already exposed to development; a second basket changes assets within the same period and broad economic environment. The comparison is consequently a bounded historical study of access and agent behavior.

The results vary by model, basket, and sampling seed. Full access improves three of four group means relative to base, but falls below equal-weight rebalancing in all four. No successful skill reads are observed, and successful algorithm calls concentrate in one model–basket–condition group. This behavior limits the interpretation of the factorial effects: they describe changed interfaces and prompts, not the benefits of financial knowledge that the model demonstrably consumed. We publish complete aggregate results, frozen protocols, source hashes, and decision receipts, keeping earlier studies separate.

Our contributions are a reusable financial skill and algorithm library; a frozen study that distinguishes document and numerical-tool access under common data and accounting; and an analy-

sis that connects returns to actual uptake, failures, risk, and computational cost. Guard detection, numerical parity, code-generation audits, and optional memory and context services provide supporting evidence in the appendices.

2 Related work

FinAgent combines multimodal financial information with trading tools, reflection, and memory (Zhang et al., 2024); StockBench evaluates sequential portfolio decisions using prices, fundamentals, and news (Chen et al., 2025). The Finance Agent Benchmark evaluates financial research tasks (Bigeard et al., 2025). Our experiment focuses on optional access to reusable documents and numerical interfaces while holding the available price history fixed. It does not compare complete trading systems or establish a ranking against these benchmarks.

Behavioral testing motivates measuring concrete capabilities beyond an aggregate score (Ribeiro et al., 2020). Backtest overfitting and selection bias (Bailey and López de Prado, 2014; Bailey et al., 2017), together with look-ahead leakage in financial agents (Li et al., 2025), motivate frozen analysis choices and explicit exposure limits. Our supporting guard studies examine specified financial errors; the trading study does not vary mandatory guard enforcement. The optional memory component adapts the mushroom-body model of Huang et al. (2024); no memory treatment is included here.

3 Library and access interventions

FIN-SKILLS supplies domain and per-library skills, quantitative algorithms, structured data adapters, and research checks. Skill documents identify task scopes, input conventions, dependencies, and dated sources. Numerical methods cover portfolio construction, signals, forecasting, volatility, and execution schedules. Structured responses distinguish computed results from missing dependencies and runtime errors. The broader architecture and interface inventory appear in Appendix I and the supporting framework description.

For the main experiment, knowledge access (K) exposes eight predeclared skills through paginated discovery and reading functions: portfolio optimization, covariance and risk, corporate actions, backtest validation, backtest overfitting, data quality, signal construction, and trend following. Numerical access (T) exposes a price-compatible menu, method descriptions, calling contracts, and execution. T therefore includes the minimal documentation needed to call its interfaces; K is additional domain material. The menu order is seeded, and neither condition prescribes a strategy. Computed weights require an explicit submission and are never adopted automatically.

Every cell has the same price readers and NumPy/pandas/SciPy execution environment. Direct package imports are disabled, so numerical-library access occurs through the assigned interfaces. Skill documents are optional resources, not inserted readings. This design measures the total effect of offering each interface, including its prompt text and discovery burden; it does not separately identify prose length or content effects.

4 Frozen factorial trading experiment

Models and paired conditions. We evaluate Qwen2.5-Coder 7B and 14B at the immutable revisions recorded in the protocol, with sampling seeds 11, 23, and 37. For each model, seed, and basket, the agent receives base access, knowledge access (K), numerical-tool access (T), or both (K+T). Each decision allows at most eight turns and 1,024 generated tokens per turn at temperature 0.1. Condition order is seeded. Each cell has 44 scheduled decisions, giving $2 \times 2 \times 3 \times 4 \times 44 = 2112$ decisions. No success or profit threshold selects the analysis population.

Assets, data, and timing. The original basket contains SPY, QQQ, EFA, EEM, TLT, IEF, GLD, DBC, NVDA, AVGO, ORLY, and FAST. These assets and their period were already used in development. The transfer basket was declared before downloading its outcomes: XLB, XLC, XLE, XLF, XLI, XLK, XLP, XLRE, XLU, XLV, XLY, and SHY. It changes the instruments while retaining the calendar and substantial exposure overlap. Both use the January 3, 2025–September 25, 2026 scoring window, with 434 sessions and available warmup history.

All cells receive the same total-return price index constructed from the frozen quotes and corporate actions, restricted to the decision date. The prompt states that the prices are already adjusted. Targets execute after the next session's close-to-close return; rebalancing opportunities occur every ten sessions. Missing or invalid submissions

retain drifted holdings. Portfolios are long-only, cash earns zero, and proportional cost is 5 bps per traded side. The ledger includes the first trade's fee and initial capital. Fixed strategy references use the same timing and cost conventions.

Availability and accounting checks. Before full inference, CPU qualification checked access restrictions, future-read denial, adjusted-price parity, algorithm execution, and valid submissions. A single development date per model and condition tested transport; a non-submission was retained rather than converted into a quality gate. Protocols, sources, data hashes, and the analysis plan were frozen before the full run. We subsequently verified all decision receipts and independently recomputed every cell's net-asset-value path with a dollar-holdings ledger. This verification adds no model decisions or selection rules.

Endpoints. The predeclared primary endpoint is cumulative net return from initial capital. The agent prompt asks for net risk-adjusted return; Sharpe and drawdown are secondary reported measures rather than substitutes chosen after observing results. For a model, basket, and seed, let B, K, T, F denote cumulative returns in the four cells. Paired effects are $F - B$, the knowledge marginal effect $[(K - B) + (F - T)]/2$, the tools marginal effect $[(T - B) + (F - K)]/2$, and interaction $F - K - T + B$, in percentage points. We average these within model and basket over the three declared seeds.

Risk, exposure, turnover, submissions, successful document reads, numerical calls, exact output adoption, failed turns, tokens, and generation time are secondary outcomes. We also reprice the same fixed decisions at 0, 10, and 25 bps without simulating a behavioral response. Circular calendar-block resampling uses lengths 10, 20, and 40, with 2,000 draws each; indices are shared across all cells and seeds within each group. The resulting 95% intervals describe dependence sensitivity on this path.

5 Trading results and observed use

Mixed return differences. Table 1 reports every group mean. Relative to base, full access changes original-basket returns by 5.70 points for 7B and -0.73 for 14B. The transfer differences are 1.69 and 4.70 points. Every full-access group falls below the equal-weight rebalancing reference (Table 7). Both positive and negative seed-level differences occur within every model–basket group (Table 2). All reported calendar-block intervals span zero (Table 6); no general performance advantage is established.

Access rarely becomes use. Across the complete batch, 0 successful skill-document reads and 31 decisions with successful numerical calls were observed (Table 3). In particular, algorithm use concentrates in the transfer 7B full-access cell; no tools-only cell makes a successful numerical call. Exact matching between submitted weights and a tool output is reported separately from calling it. The zero-read result makes interpreting K as consumed financial knowledge untenable. The return contrasts estimate offered access, including changes to prompts and behavior.

Failures, exposure, and costs. Submission denominators include all scheduled decisions, and failed turns remain in the record. A missed rebalance can leave a profitable or losing position in place, so returns do not directly measure interface competence. Tables 4 and 5 report exposure, drawdown, turnover, token use, generation time, and transaction-cost sensitivity. Appendix A and the aggregate evidence retain all seed-level results; none are selected for successful tool use or positive return.

6 Discussion

The main finding is a gap between resource availability and resource use. Scripted qualification established that the selected numerical methods could execute and submit valid weights. The agents nevertheless rarely called them and never successfully read a skill document. A favorable return in a knowledge-access cell therefore cannot establish that financial knowledge improved the strategy. Similarly, higher submission rates can coexist with lower return; reaching a valid output and making a profitable allocation are different outcomes.

The return differences are sensitive to model, basket, and sampling seed. The original 7B tools-only condition has the highest group mean on that basket, but its full-access condition performs worse. The original 14B tools-only mean falls below base, while the transfer full-access means improve for both model sizes. These patterns do not support a stable ordering of the four interfaces.

Basket	Model	Base	K	T	K+T	Full−Base	K effect	T effect	Interaction
Original	7B	28.10	34.73	54.56	33.80	5.70	-7.07	12.76	-27.39
Original	14B	40.65	44.91	28.95	39.92	-0.73	7.62	-8.35	6.71
Transfer	7B	10.73	15.17	13.20	12.42	1.69	1.83	-0.14	-5.21
Transfer	14B	9.67	10.21	4.70	14.37	4.70	5.11	-0.41	9.13

Table 1: Mean cumulative net return (percent) and paired effects (percentage points), over three sampling seeds on the same historical calendar. All planned decisions, including missing submissions, contribute. K denotes skill-document access; T denotes numerical-tool access with calling contracts. Original assets were used in development; transfer changes assets within the same period.

Common-period asset transfer adds a useful check on asset dependence while retaining shared market direction and exposures.

Research guards answer a separate reliability question. The supporting audits test known defects, numerical parity, and whether an enforced controller rejects erroneous programs. Their results cannot explain the trading return differences because guard enforcement is not a factor in this experiment. The retained code-generation study also shows that rejecting wrong programs need not produce more correct programs. Together, these studies motivate reporting availability, use, acceptance, and outcome separately.

7 Conclusion

A frozen four-condition experiment of FINSKILLS finds mixed historical return effects and limited voluntary uptake of financial documents and algorithms. Full access improves three group means against base but none against equal-weight rebalancing; the reported dependence-aware intervals all span zero. The released evidence supports a bounded account of what these models did when interfaces were offered. It does not establish a general advantage for tool-equipped trading agents or a causal benefit from knowledge consumption. The library's separate verification functions remain useful capabilities whose effects must be tested with their own interventions.

Limitations

The original assets were exposed during development. Transfer assets were selected before this batch's outcomes were downloaded, but share the same dates and broad economic exposures; they are not an independent market or prospective test. Model pretraining contamination is not ruled out. Frozen present-day skill documents are not claimed to have been available at the historical decision dates. The two models are sizes of one family, and three sampling seeds share one market path. Neither decisions nor seeds are treated as independently sampled markets.

The study is long-only with zero cash yield, a fixed proportional transaction cost, and close-price execution. It does not model executable quotes, capacity, additional slippage, or market impact. Block resampling describes sensitivity within the observed path, not out-of-market validity. Rare tool use prevents identifying benefits among users, and no counterfactual removal of individual calls is performed. Costs, errors, and non-submissions remain in the analysis rather than selecting successful episodes. The supporting synthetic guard and code-generation tasks have their own exposure and coverage limits, reported with those studies. Human productivity and prospective trading performance remain unmeasured.

Ethics and Reproducibility

The experiments are simulated and place no live orders. The release includes experiment code, the frozen protocol and analysis plan, aggregate results for every cell and seed, and source, data, and decision hashes. Market datasets, model weights, and raw model outputs remain on the controlled research host and are not redistributed in this release. Readers can verify the published aggregate tables and their receipt linkage; reproducing inference requires the recorded model revisions and separately obtained market data. This working manuscript is not a claim of venue submission or completed authorship review.

References

David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2017. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69.

David H Bailey and Marcos López de Prado. 2014. The deflated Sharpe ratio: Correcting for selection bias,

backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*.

Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*.

Cheng Huang, Junjie Luo, Seung Je Woo, Lucas A. Roitman, Jizhou Li, Vincent A. Pieribone, Madhuvanthi Kannan, Ganesh Vasan, and Mark J. Schnitzer. 2024. Dopamine-mediated interactions between short- and long-term memory dynamics. *Nature*, 634:1141–1149.

Xiangyu Li, Yawen Zeng, Xiaofen Xing, Jin Xu, and Xiangmin Xu. 2025. Profit mirage: Revisiting information leakage in llm-based financial agents. *arXiv preprint arXiv:2510.07920*.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*.

Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. *arXiv preprint arXiv:2402.18485*.

A Complete factorial results

The following tables retain every group and sampling seed. All values are generated from the verified aggregate evidence. The repository also preserves separate execution and delivery failures, complete per-seed risk and exposure measures, source and protocol hashes, and the decision receipt map. No failed decision or unfavorable cell is excluded.

Basket	Model	Seed	Base	K	T	K+T	Full−Base	Interaction
Original	7B	11	39.08	28.89	41.66	39.97	0.89	8.50
Original	14B	11	50.41	49.17	34.75	41.29	-9.12	7.77
Original	7B	23	20.66	36.82	56.21	14.44	-6.23	-57.92
Original	14B	23	32.39	29.55	16.57	20.86	-11.53	7.13
Original	7B	37	24.56	38.49	65.81	46.98	22.42	-32.75
Original	14B	37	39.15	56.01	35.52	57.61	18.46	5.23
Transfer	7B	11	6.78	18.23	7.04	10.72	3.95	-7.76
Transfer	14B	11	21.14	7.35	-2.70	30.19	9.05	46.67
Transfer	7B	23	17.54	17.81	22.29	10.13	-7.41	-12.44
Transfer	14B	23	-1.04	6.99	7.69	8.01	9.05	-7.71
Transfer	7B	37	7.87	9.46	10.26	16.41	8.54	4.57
Transfer	14B	37	8.92	16.30	9.11	4.91	-4.00	-11.58

Table 2: All seed-level cumulative net returns (percent) and paired effects (percentage points). Each cell retains all 44 decisions. Seeds vary sampling, not market history.

Basket	Model	Cell	Submitted	Alg. decisions	Matched	Skill reads	Failed turns
Original	7B	base	65/132	0	0	0	594/668
Original	7B	knowledge	98/132	0	0	0	375/488
Original	7B	tools	111/132	0	0	0	356/521
Original	7B	full	95/132	0	0	0	444/576
Original	14B	base	130/132	0	0	0	90/220
Original	14B	knowledge	132/132	0	0	0	108/240
Original	14B	tools	128/132	0	0	0	123/256
Original	14B	full	115/132	1	0	0	333/450
Transfer	7B	base	47/132	0	0	0	756/808
Transfer	7B	knowledge	70/132	0	0	0	600/696
Transfer	7B	tools	105/132	0	0	0	393/549
Transfer	7B	full	99/132	29	21	0	337/583
Transfer	14B	base	120/132	0	0	0	240/372
Transfer	14B	knowledge	126/132	0	0	0	156/283
Transfer	14B	tools	128/132	0	0	0	128/261
Transfer	14B	full	102/132	1	1	0	437/553

Table 3: Observed uptake and failure denominators, summed across all three seeds. Algorithm decisions count decisions with at least one successful numerical call; matched counts submissions whose weights equal a successful tool output within the frozen tolerance. Skill reads count successful calls, not unique documents. Failed turns include execution and response delivery failures; the disaggregation is preserved in analysis.json. No-use is an outcome.

Basket	Model	Cell	Sharpe	Max DD (%)	Invested (%)	Largest (%)	Turnover
Original	7B	base	0.83	-22.72	98.46	21.19	12.83
Original	7B	knowledge	1.04	-15.45	99.12	23.18	26.82
Original	7B	tools	1.53	-13.21	97.66	41.62	37.62
Original	7B	full	0.96	-19.09	97.69	32.16	34.96
Original	14B	base	1.13	-12.35	98.59	32.01	41.49
Original	14B	knowledge	1.28	-13.16	99.43	30.96	40.51
Original	14B	tools	0.74	-16.46	99.23	43.43	39.46
Original	14B	full	1.17	-12.63	98.46	30.67	39.27
Transfer	7B	base	0.73	-13.11	83.63	41.14	13.96
Transfer	7B	knowledge	0.75	-13.34	97.72	27.89	17.10
Transfer	7B	tools	0.91	-9.57	97.82	68.01	36.94
Transfer	7B	full	0.65	-11.28	98.91	53.57	34.88
Transfer	14B	base	0.51	-12.94	97.95	44.67	38.07
Transfer	14B	knowledge	0.59	-11.40	99.45	45.52	42.58
Transfer	14B	tools	0.26	-15.83	99.23	62.32	43.33
Transfer	14B	full	0.96	-10.17	97.69	43.38	35.75

Table 4: Means of seed-level risk and exposure measures. Sharpe uses daily returns, annualization by 252 sessions and zero cash yield. Max DD is signed maximum drawdown; invested and largest are mean daily weights. Turnover is the sum of absolute risky-asset weight changes across trades (one-way units, not percent). These are descriptive, unadjusted comparisons.

Basket	Model	Cell	0 bps	10 bps	25 bps	Input M	Output M	Gen. hours
Original	7B	base	28.92	27.28	24.87	2.14	0.38	2.50
Original	7B	knowledge	36.55	32.93	27.68	1.34	0.24	1.60
Original	7B	tools	57.49	51.68	43.36	1.33	0.29	1.89
Original	7B	full	36.13	31.50	24.85	1.73	0.31	2.02
Original	14B	base	43.59	37.77	29.47	0.34	0.12	1.46
Original	14B	knowledge	47.86	42.02	33.69	0.33	0.12	1.45
Original	14B	tools	31.51	26.43	19.16	0.57	0.15	1.89
Original	14B	full	42.69	37.20	29.34	1.46	0.25	3.22
Transfer	7B	base	11.50	9.96	7.70	2.59	0.46	3.03
Transfer	7B	knowledge	16.15	14.19	11.30	2.18	0.37	2.42
Transfer	7B	tools	15.32	11.11	5.09	1.69	0.31	2.06
Transfer	7B	full	14.40	10.47	4.82	1.74	0.30	1.99
Transfer	14B	base	11.79	7.59	1.58	1.03	0.24	3.01
Transfer	14B	knowledge	12.58	7.90	1.22	0.59	0.16	2.00
Transfer	14B	tools	7.00	2.45	-4.04	0.61	0.15	1.91
Transfer	14B	full	16.40	12.38	6.61	1.86	0.33	4.18

Table 5: Fixed-decision cost sensitivity: mean return (percent) repriced at 0, 10 and 25 bps per traded side; the primary analysis uses 5 bps. Tokens and model generation hours are summed across seeds (not elapsed wall time or API prices). Cost sensitivity does not simulate changes in agent behavior. Full precision and separate failure counts are in the aggregate evidence.

Basket	Model	Effect	10 sessions	20 sessions	40 sessions
Original	7B	full minus base	[-21.01, 32.53]	[-19.25, 32.79]	[-17.10, 29.17]
Original	7B	knowledge main	[-25.05, 12.51]	[-24.20, 13.35]	[-23.71, 11.39]
Original	7B	tools main	[-9.96, 32.04]	[-10.84, 30.30]	[-9.52, 29.76]
Original	7B	interaction	[-67.23, 19.40]	[-77.84, 19.70]	[-89.26, 25.83]
Original	14B	full minus base	[-36.77, 28.65]	[-32.56, 27.38]	[-27.73, 23.31]
Original	14B	knowledge main	[-19.37, 28.81]	[-18.87, 28.43]	[-13.40, 29.54]
Original	14B	tools main	[-28.03, 10.70]	[-28.33, 8.69]	[-27.94, 8.26]
Original	14B	interaction	[-36.84, 39.52]	[-30.52, 33.52]	[-25.98, 29.72]
Transfer	7B	full minus base	[-15.26, 18.03]	[-15.08, 17.94]	[-16.69, 19.20]
Transfer	7B	knowledge main	[-10.94, 14.97]	[-10.41, 13.95]	[-8.99, 12.67]
Transfer	7B	tools main	[-13.44, 11.11]	[-13.30, 10.90]	[-13.26, 11.54]
Transfer	7B	interaction	[-25.92, 12.75]	[-25.06, 14.12]	[-18.32, 9.68]
Transfer	14B	full minus base	[-10.63, 18.70]	[-10.26, 18.88]	[-10.26, 20.14]
Transfer	14B	knowledge main	[-6.01, 16.92]	[-4.68, 14.91]	[-3.21, 13.98]
Transfer	14B	tools main	[-11.53, 11.06]	[-11.74, 10.24]	[-10.59, 10.70]
Transfer	14B	interaction	[-11.44, 30.25]	[-11.43, 27.40]	[-13.56, 27.32]

Table 6: Descriptive 95% paired circular calendar-block intervals (percentage points), 2,000 draws for each length. Calendar indices are shared across cells and seeds in a group. These intervals do not represent independent markets or establish general performance gains.

Basket	Fixed strategy	Return (%)	Sharpe	Max DD (%)	Turnover
Original	equal weight	42.13	1.52	-12.19	2.21
Original	inverse volatility	31.14	1.66	-8.13	2.15
Original	min variance	11.70	1.41	-3.64	2.65
Original	hrp	15.09	1.26	-5.89	4.50
Original	equal weight hold	39.45	1.43	-11.49	1.00
Original	cash	0.00	0.00	0.00	0.00
Original	60 40 rebalance	20.88	1.10	-10.53	1.47
Transfer	equal weight	22.32	1.03	-13.65	1.89
Transfer	inverse volatility	12.49	1.07	-7.12	1.84
Transfer	min variance	6.06	2.45	-0.89	1.22
Transfer	hrp	5.78	2.11	-1.01	1.42
Transfer	equal weight hold	21.83	1.01	-13.64	1.00
Transfer	cash	0.00	0.00	0.00	0.00

Table 7: Fixed strategy references use the same next-close timing and 5 bps cost. Portfolio methods use 252 past returns and frozen defaults. Equal-weight hold trades once; other methods rebalance every ten sessions. The original-only 60/40 is SPY/IEF. Cash earns zero.

B Earlier bundled-access trading study

Before the factorial experiment, a separate development study compared a bundled library interface against a raw-data interface on the original basket. It retained all 528 scheduled decisions, including failures and missing submissions. The bundle changed data adapters, documentation, and numerical tools together; its contrast cannot identify the contribution of any one component and is not pooled with the factorial cells.

The complete positive and negative results, seed-level risk and cost metrics, failure counts, and post hoc exposure and adoption analysis are archived separately in `evidence/20261002-trading-v5`. A Slurm memory failure occurred after one pair's complete inference receipt had been written; CPU scoring subsequently verified those saved decisions without new inference. The archive preserves this recovery record.

Model	Seed	Raw (%)	Bundle (%)	Difference (pp)	Raw submit	Bundle submit	Alg. decisions
7B	11	3.44	38.04	34.60	27/44	19/44	1
7B	23	31.63	43.98	12.35	29/44	17/44	1
7B	37	20.68	29.11	8.43	32/44	24/44	2
14B	11	35.06	23.93	-11.13	44/44	36/44	1
14B	23	39.52	34.21	-5.31	42/44	44/44	3
14B	37	37.52	24.95	-12.58	42/44	38/44	2

Table 8: Earlier development-exposed bundled-interface study, retained separately from the factorial batch. All scheduled decisions are scored, and all seed-level signs are retained. The bundle changes data adapters as well as tools and documentation. Full risk, failure, token, cost, and post hoc exposure/adoption results accompany the separate evidence archive.

C Library architecture: knowledge, computation, and verification

FIN-SKILLS connects three reusable layers. Skills describe financial assumptions and library conventions. Numerical interfaces execute methods against declared inputs. Research guards return structured findings that a controller can require before accepting an artifact. A successful algorithm call alone does not imply that the required audit ran.

Guarded research workflow. For a task and its available data, an agent selects a skill, inspects a numerical interface, and proposes an artifact. A controller executes the relevant checks on that artifact. Failures return actionable findings; missing required evidence remains incomplete. The controller accepts only artifacts meeting its declared policy and saves the associated execution records. This workflow is distinct from offering an agent optional tool access and trusting its claim to have validated the output.

Optional experience services. HiSTrim operates within a single session by evicting tokens from the KV cache during generation. Persistent memory operates across sessions by storing parameter updates in an external module. The two mechanisms operate on different timescales: context pruning controls what the model attends to right now, whereas persistent memory retains summary statistics of what happened in previous episodes. Neither service is required to run the library's guards or reproduce its numerical primitives.

Grounded feedback updates. When an episode concludes, the agent records the market context, the chosen tool, and the realized return. Only verified, settlement-cleared outcomes update memory. Negative returns are retained as learning signals, while trades relying on look-ahead data or unadjusted prices are rejected by guards to prevent memory corruption.

D Execution and feedback verification

Tool execution and memory updates require verifiable records of what actually ran. The execution controller computes cryptographic hashes over input data, executed code, and returned allocations. If an agent modifies an artifact after execution, previous validation receipts become invalid. The controller returns `PASS` if all required guards pass, `FAIL` if a guard detects a defect, and `INCOMPLETE` if evidence is missing. This mechanism enforces execution provenance without modifying the underlying algorithm.

For cross-episode memory, timing and accounting checks verify that delayed outcomes cleared settlement before they update persistent state. Trades that rely on look-ahead data or unadjusted corporate splits are blocked, preventing corrupted feedback from distorting the agent's memory. Unsettled trades wait for settlement, while valid unprofitable trades remain eligible learning signals.

The audit study evaluates models under unconstrained generation, optional self-checking, and mandatory verification. Appendix H describes the evaluation setup, defect detection benchmarks, and empirical audit results across model tiers.

E Library evaluation: defects, execution, and numerical parity

We evaluate the financial library at three boundaries: detecting invalid research inputs, executing declared interfaces, and reproducing numerical references; Section F adds the code-generation audit (E4). Table 10 summarizes the observed evidence. Each row has its own unit and denominator.

E1: detecting research defects. The benchmark plants 12 defect families, including temporal leakage, unadjusted corporate actions, and universe-selection errors, in three synthetic worlds. The suite records 36/36 detections and no false alarms on its clean references. This is suite-level recall: a defect may be caught by another applicable guard even when one intended check misses it. Repeating a family across seeds does not create a new defect type. Appendix H describes the independent oracle and the distinction between diagnosis and agent repair.

E2: execution and audit compliance. We count actual dispatches and final artifacts rather than statements that a check ran. The exploratory portfolio study completed all 64 decisions, including 16 with autonomous tool access, and logged 43 successful requests in that arm. These requests include catalog lookup and guidance retrieval as well as numerical algorithms. In the separate audit feasibility study, optional guards and mandatory enforcement have different acceptance outcomes. Mandatory rejection protects the submis-

Component	Role in research workflow	Target question	Implementation and validation
Financial skills	Route tasks and document dated conventions	What assumptions and failure modes apply?	129 skill documents
Numerical interfaces	Discover and execute quantitative methods	Do selected computations agree with references?	48 catalog entries; E3 tests eight primitives
Executable guards	Diagnose data, accounting, and validation defects	Which required checks passed, failed, or lack inputs?	36 registered guards; E1 exercises 13
Execution records	Bind verification to the submitted artifacts	Did the required checks actually run on this code and data?	Explicit pass, fail, and incomplete states
Optional extensions	Select context and retain verified experience	How do auxiliary context and memory services affect tasks?	HiSTrim and persistent memory; separate studies

Table 9: Core financial library components and optional context and memory integrations.

Study	Endpoint	Observed result	Scope
E1: guards	Planted-defect detection	36/36 detected (100%); no clean-reference false alarms	12 defect families in three seeded worlds; 13 core guards exercised
E2: execution	Valid decisions and successful tool requests	64/64 valid decisions across four conditions; 43 successful autonomous tool requests	16 autonomous decisions; requests include discovery, reading, advice, and computation
E2: audit	Artifact validation and reporting fidelity	Reference submission accepted; pilot Opus Sharpe gap 0.188 to 0.034	Historical pilot; self-reported guard use is not an execution receipt
E3: parity	Agreement with numerical references	8/8 checks within case-specific tolerances	Selected primitives; maximum absolute error 1.18×10^{-8}
E4: code	Valid programs; accepted but wrong	First submission 0/48 (1/48 post hoc). Enforced, three tries: valid 1, 1, 0 of 24; accepted but wrong 22, 2, 0	Two model sizes, four rules, three seeds; arms: raw, raw with checks, library with checks

Table 10: Core library evidence. Detection, executable completion, audit acceptance, and numerical agreement are different endpoints. No row establishes universal correctness or profitable trading.

sion boundary but does not constitute successful repair: the 14B model reached its turn limit in all three mandatory cells. The reference submission passed the public checks and independent grading.

E3: numerical parity. Eight checks compare equal weighting, inverse-volatility weighting, minimum-variance optimization, normal and historical VaR/ES, historical volatility, fractional-difference weights, and an augmented Dickey–Fuller statistic with closed forms or independent NumPy, SciPy, pandas, and statsmodels references. All meet their recorded tolerances; the largest absolute difference is 1.18×10^{-8} for the optimization weights. The optimization case allows 10^{-3} while other cases have their own tighter tolerances. This supports agreement for the tested calculations, not exact equality or validation of all catalog entries.

Auxiliary integrations. The context and memory studies test optional consumers of the library's results. Their methods and outcomes appear in the appendices. The public FinQA RAG and reranking runs retain their original official scores (4/192 correct programs across six model and retrieval cells). A post-hoc reanalysis bound its target-blind extractions by hash before grading: accepting a single fenced answer adds no correct program, and counting explicit correct final numbers raises the total to 7/192. That second count is not program accuracy. Of the 96 Mistral answers, 75 have no final answer, so format repair alone cannot isolate retrieval quality (Appendix I). Interface corrections require a new versioned comparison rather than retrospective replacement of the recorded grades.

F E4: Financial code generation with and without enforcement

Can an open code model write a correct allocation or signal rule from a specification alone, and does the library's documentation, or enforcing its use and checks, change that? We froze a paired first-submission study before model inference. Qwen2.5-Coder 7B and 14B each wrote one program for four rules (diagonal risk parity

by inverse volatility, positive cross-sectional momentum, a 5/20 moving-average crossover, and volatility-targeted momentum), three data seeds, and two arms: 48 programs. Both arms received identical raw quotes with split jumps, cumulative split factors known on their event dates, the mathematical rule, the one-bar timing rule, one response of at most 2,048 tokens, and no execution feedback. The library arm additionally received the actual catalog output and interface and guard documentation (mean prompt 11,667 versus 10,756 tokens). Programs ran in the Landlock/seccomp worker. An independent NumPy grader, shared by both arms, requires execution, agreement with the reference within 10^{-8}, unchanged positions through session k when prices at and after k are perturbed ($k = 32, 64$), and identical output for the economically equivalent split-adjusted input. A program is valid only if it passes all.

Pre-registered outcome. No program was valid in either arm (0/48). The protocol asked for Python only, optionally inside one fence. In all, 37 responses appended an explanation after the fence (24 from 7B and 13 from 14B), so the frozen parser compiled prose and recorded a `SyntaxError`. The 11 executed programs all failed numerical agreement and split invariance. No response was truncated. None of the 24 library-arm responses imported or mentioned the library, and no algorithm or guard call was traced. The comparison therefore measures documentation exposure without uptake; it does not estimate the effect of using FIN-SKILLS.

Post-hoc extraction and failure modes. To separate formatting from computation, we froze an arm-blind extraction of the first complete fence by hash before execution and reran the same worker, sandbox, and grader on another Linux host; it reproduced all 11 official executions exactly. Then 37 programs execute, but only 1 of 48 is valid. The failures are silent in the sense that matters for research: 24 of 24 14B programs, in both arms, applied a cumulative product to split factors that the prompt defines as already cumulative, and 26 executed programs change their positions when the same prices are presented split-adjusted. In 15 executed programs a position moves when prices at or after its own session change; 12 use the same-session close and 3 use later data. The 7B programs that still fail raise shape, index, or pandas-API errors.

What the library guard observed. Run post hoc on each executed program, the library's causality guard flagged all 3 programs with strictly-future dependence and none of the 22 programs that passed the grader's timing probes. It passed the 12 same-session violations: the guard asks whether values before k depend on data at or after k, which is feature causality, whereas a position must also exclude session k itself. The execution-lag contract therefore needs its own check rather than an inference from a passed causality test. Together with E2, these results show that raw first submissions are unreliable in ways that still execute, and that optional documentation is not used.

Enforced library calls and checks. We therefore froze a follow-up on the same rules, seeds, models, prompt packet, and hidden grader, with up to three attempts and controller feedback in three arms of 24 episodes: coding from scratch with execution feedback only; coding from scratch with enforced lag and split checks; and the library documentation with an enforced `fin_skills.algorithms.run` call plus the same checks. The lag check applies the library's causality guard to positions shifted up one row ($k = 40, 72$), closing the same-session gap above; the split check requires identical output for the split-adjusted representation. Feedback never reveals reference weights, and every arm parses the first complete fence. A deliverable counts only if the controller accepts it and the unchanged grader finds it valid.

Enforcement did not produce more valid programs (Table 12): 1, 1, and 0 of 24. It changed what is delivered. With execution feedback alone, 23 programs were accepted and 22 of them were wrong (91.7%), including 12 split and 9 timing failures. With enforced checks, accepted-but-wrong programs fell to 2 (8.3%), both of which pass the grader's timing and split probes and fail only numerical agreement; with the library requirement, none was accepted. The library was hard to use from its interface text: 42 of 72 library-arm attempts reference FIN-SKILLS, 8 trace a call, and 51 fail to run, mostly on output shape, nonfinite values, and argument errors. In 7 attempts the model followed a parameter name in the supplied interface text that the function does not accept; all 7 also re-cumulated split factors and would have failed the split check. For these models and this

		Executed		Post-hoc extraction				Mean per response	
Model	Arm	Official	Post-hoc	Correct	Split fail	Timing fail	Valid	Output tok.	Gen. s
7B	Raw	0/12	7/12	1	2	2	1/12	398	8.4
7B	Library	0/12	6/12	0	0	4	0/12	337	7.8
14B	Raw	5/12	12/12	0	12	6	0/12	312	9.4
14B	Library	6/12	12/12	0	12	3	0/12	286	9.0

Table 11: Paired first-submission code generation. Under the pre-registered parser no program is valid in any cell (0/48). Post-hoc columns take the first complete code fence; failure counts are among executed programs. No library-arm program called a library algorithm or guard.

Arm	Valid	Acc.	Wrong	Lib.	Tries
Raw	1/24	23	22	—	1.25
Raw + checks	1/24	3	2	—	2.79
Lib. + checks	0/24	0	0	5/24	3.00

Table 12: Enforced follow-up (job 1765432; 7B and 14B pooled, up to three attempts). Raw receives execution feedback only. Valid: accepted and valid under the hidden grader. Acc.: accepted. Wrong: accepted but invalid. Lib.: episodes with a traced library call in any attempt. Tries: mean attempts.

Arm	Valid	Acc.	Wrong	Lib.
Raw	1/24	22	21	—
Raw + checks	2/24	4	2	—
Lib. + checks	0/24	0	0	9/24

Table 13: Complete v2 follow-up. Valid, Acc., and Wrong match Table 12; Lib. counts final submissions with a traced call (rather than any attempt). The library-arm intervention bundles documentation, rule cards, required use, and checks.

budget, the checks turn silent errors into explicit rejections. These results alone do not identify whether model capability, repair budget, or the interface is the main obstacle to correct construction.

Completed interface-repair follow-up (v2). An October 29 follow-up (job 2906547) retained the same tasks, seeds, models, raw-arm prompts, and numerical grading equations. Its library arm corrected argument names, documented the panel-level interface, and added rule cards and explained check feedback. An additional strict-lag audit is recorded as a diagnostic, separately from the frozen valid-deliverable endpoint. These changes bundle interface and feedback revisions, so the two versions are not a randomized comparison of a single repair. All 72 planned episodes are included in Table 13.

The revised library arm has 9 final submissions with a traced call (10 episodes across any attempt), but no valid deliverables. Checks reduce accepted errors without establishing successful repair. The task families were already exposed during interface development; these are regression results from two sizes of one model family, not an independent generalization test. Subsequent fixes to double-adjustment handling and the task execution boundary have only local scripted qualification in this manuscript. They do not change either version's frozen outcomes or demonstrate an improvement in model-generated programs. A proposed self-improvement benchmark is a separate, unrun study and contributes no result here.

G Component ablation: algorithm access without mandatory guards

This exploratory study tests a restricted algorithm-access component of the library. The dispatcher offers discovery, skill reading, computation, and optional advice, but does not enforce the library's research guards on final allocations. We treat it as an unguarded component ablation, not as a test of the complete guarded workflow. No matched guarded arm was run in this experiment.

G.1 Experimental setup

We test Qwen2.5-Coder 7B and 14B under four conditions: historical prices alone; prices with static skill excerpts; prices with precomputed tool outputs; and autonomous access to the library's algorithm catalog. Each condition runs across inference seeds 11 and 23, with a budget of five interaction turns per decision (capped at 256 generated tokens per response). Conditions run in randomized order.

In the autonomous condition, the agent receives descriptions of available tools. It can inspect algorithm specifications, run an algorithm with custom arguments, or query a rule-based strategy adviser. Tool calls are dispatched against trailing market snapshots and return structured numerical

Condition	7B	14B
No library	$+0.100$	-3.324
Skill text	-2.930	-4.567
Fixed tool outputs	-3.075	-3.030
Autonomous library	-3.984	-3.783

Table 14: Mean cumulative proxy return (%) across inference seeds with 5 bps transaction costs per side.

results. In our evaluation, agents access 18 price-compatible quantitative routines.

A decision without valid portfolio weights leaves existing allocations unchanged. Requests, outputs, and intermediate reasoning traces are logged separately, and final weights are frozen before scoring.

G.2 Data and evaluation protocol

We evaluate on daily exchange rates for EUR against USD, JPY, GBP, and CHF from ECB reference data. Messages use anonymized asset identifiers and relative indices rather than calendar dates.

Each decision observes 126 trailing days. The evaluation spans the first 84 trading days of 2025 (ending May 2), with four rebalancing points spaced 21 days apart. Decisions at step t execute at day $t + 1$ prices to prevent look-ahead bias. Portfolios are long-only without leverage, with unallocated capital held in cash. We charge 5 basis points per side for rebalancing and final liquidation. Cash, equal-weight buy-and-hold, and deterministic rules serve as reference baselines.

G.3 Observed execution and returns

Across 16 trials, models generated 64 valid final portfolio allocations. The autonomous conditions account for 16 decisions and 43 successful requests: 13 from 7B and 30 from 14B. Requests include catalog discovery, skill reading, strategy advice, and algorithm execution; they are not 43 numerical algorithm executions. The completed autonomous conditions recorded no tool or format errors. An earlier parser-failure attempt is retained separately and is not included in these completed trials.

Tool choices diverged across model scales: 7B selected cross-sectional momentum, Bollinger reversion, and volatility-targeted momentum, whereas 14B predominantly used Bollinger reversion alongside trend-following indicators.

Table 14 shows returns across all arms. While autonomous 14B improved over its text-only condition by 0.783 percentage points, neither autonomous model beat the unassisted baseline (+0.100% for 7B, -3.324% for 14B).

Recorded portfolio holdings show different exposure choices across conditions: mean risky asset exposure for 7B rose from 0.500 without the library to 0.969 with autonomous tools; for 14B, exposure shifted from 1.000 (all-in) to 0.500. Autonomous runs used 38 model calls per model across eight decisions, totaling 1,155,782 prompt tokens and 10,165 completion tokens.

The valid decisions and successful requests provide interface evidence for E2. The returns describe one short, anonymized currency path with transaction costs. They do not establish that the financial library is ineffective, nor that missing guard enforcement caused the losses: the experiment lacks the paired intervention required for either conclusion. A full-workflow comparison must hold data, models, budgets, execution rules, and costs fixed while varying mandatory guards. Memory and context pruning are separate interventions and are not inferred to remedy these returns.

H Supporting audit and validation studies

This appendix details the execution and feedback validation checks, defect recall benchmarks, and empirical audits across model tiers.

H.1 Independent evaluation environment

We evaluate agent portfolio decisions in a synthetic market environment with full ground-truth transparency, providing quoted prices, corporate split actions, delisting events, and fundamental disclosures. A submitted strategy script returns portfolio weights, alongside a written report stating its expected Sharpe ratio. Session-t portfolio weights must use only information available by the close of session $t-1$.

An independent oracle evaluator verifies submissions by perturbing future data, checking for post-delisting exposure, and recalculating performance under fixed transaction costs. Oracle grades are generated after the submission receipt is saved and are never exposed to the agent during its decision turns.

The oracle tests whether allocations depend on unavailable information. The recorded latent-drift reference earns a sample net Sharpe of 1.63 with a one-day lag; this is a reference strategy, not a proved ceiling on attainable performance. Wrong-side joins can inflate reported performance, but a high Sharpe alone does not diagnose leakage. Dependency perturbations and availability-time checks establish the defect.

H.2 Validation benchmarks

Defect detection benchmark. We evaluate 13 core research guards against 12 planted quantitative defect families, including look-ahead feature joins, unadjusted stock split prices, future earnings disclosure leaks, and survivorship bias. Across three seeded synthetic market worlds, the guards achieve 100% recall (36/36 planted defects caught) with zero false alarms on clean reference code.

Reporting discrepancy and compliance paradox. Table 15 measures reporting discrepancy: the absolute difference between the Sharpe ratio claimed by the agent in its written report and the ground-truth Sharpe ratio computed by the oracle evaluator.

Without tools, Opus exhibited a reporting gap of 0.188, which decreased to 0.034 when given execution guards (81.9% reduction). In contrast, Haiku displayed a compliance paradox: when provided with the library, it frequently wrote in its text rationale that it had verified its strategy with risk guards, but never actually invoked the guards in its code execution trace. This superficial compliance inflated its discrepancy to 11.949, demonstrating that smaller models require automated code execution verification rather than relying on self-reported compliance.

Model and condition	Mean absolute Sharpe gap
Opus, no library	0.188
Opus, with library	0.034
Haiku, no library	0.574
Haiku, with library	11.949

Table 15: Reporting discrepancy between agent-claimed Sharpe ratios and ground-truth oracle evaluations.

Social sentiment verification. On 1.72 million financial posts across 2,521 social media accounts, raw uncalibrated sentiment scores exhibited negative out-of-sample rank correlation with subsequent asset returns (Rank IC $= -0.0318$). By evaluating each account's historical prediction accuracy and applying Bayesian credibility weighting (including inverting signals from consistently inaccurate accounts), the out-of-sample correlation was reported as $+0.0104$. These values are retained from the supplied summary fixture; row-level predictions and timestamp provenance were unavailable for independent reproduction. They are not verified trading-return results or core library evidence.

H.3 Four-condition audit feasibility

We evaluated models under four audit conditions: unassisted baseline (C0), text guidance only (C1), optional executable guards (C2), and mandatory automated audit (C3).

Operational findings. Table 16 presents results for Qwen2.5-Coder 14B under automated audit constraints. The acceptance rate was 67% under C0 and C1, 100% under C2 (optional guards), and 0% under C3 (mandatory audit), where runs reached the turn limit while attempting to fix all detected code defects. Mean token consumption under C3 was 47,491 compared to 13,877 under C2. This records additional work without successful acceptance; token consumption alone does not

establish better debugging.

A reference quantitative script passed all verification and timing checks on the market benchmark, confirming the end-to-end viability of the automated verification pipeline.

Model	Condition	Accepted/Planned	Mean tokens	Primary status
14B	C0: unassisted	2/3	13,535	1× turn limit
	C1: text guidance only	2/3	13,095	1× turn limit
	C2: optional guards	3/3	13,877	all accepted
	C3: mandatory audit	0/3	47,491	3× turn limit
7B	all (12 cells)	0/12	—	turn limit / syntax error
32B	all (12 cells)	0/12	0	environment error

Table 16: Audit feasibility results across open-weight models under automated verification constraints.

Component	Entries
Skill documents	129
Algorithm–backend catalog	48
Registered research guards	36
JSON-callable tools	56

Table 17: Inventory of quantitative research library interfaces.

I Library implementation and interface scope

Knowledge and executable interfaces. Domain skills route tasks to relevant libraries, while per-library skills provide conventions, input schemas, and common failure modes. The Python package exposes catalog search, skill loading, and reference retrieval. Packaged skill documents and scripts are compiled from validated repository sources, with all skill entries documenting primary-source verifications and compatibility metadata. Table 17 summarizes the core library components.

Algorithm selection and execution. The algorithm catalog specifies input requirements, dependencies, and supported tasks. Implemented families include portfolio construction, forecasting, volatility and risk estimation, technical signals, and execution schedules. Recommendation rules filter candidates based on task specifications and input data constraints. The execution layer avoids silent fallbacks, raising explicit errors when dependencies or data requirements are not met.

Reusable models and lifecycle. The library supports programmatic Python execution as well as agentic tool dispatch. Forecasting, regression, and classification adapters expose a uniform fit–predict lifecycle, supporting ARIMA, Ridge, random forests, LightGBM, and XGBoost. Fitted models and estimators can be serialized and restored alongside environment metadata. The model registry additionally provides an adapter for the bio-inspired memory architecture via `create_model("fly_memory")`, supporting prediction, verified feedback updating, and state serialization.

Built-in retrieval and model composition. The RAG interface chunks packaged skills or external documents and retrieves passages using BM25 and neural rerankers. Passages retain document identifiers, source offsets, and availability timestamps; point-in-time retrieval excludes documents unavailable at the requested decision cutoff. Citation checks validate references against retrieved passage identifiers. The pipeline freezes candidates, temporal cutoffs, and context budgets prior to neural reranking.

Information availability and point-in-time processing. News and disclosure collection supports RSS/Atom feeds, static pages, GDELT, and official regulatory sources. Collection records distinguish publication time from observation time and retain revision histories. Point-in-time digests filter future, stale, and undated records while deduplicating canonical URLs.

Model choice and rule-based advice. An agent can inspect skills, discover candidate algorithms, execute chosen methods with custom parameters, and incorporate numerical outputs into its decision logic. Market-state categorization provides heuristic regime indicators (trend, range, stress) that agents can inspect as additional evidence alongside price tables.

Research workflow and audit boundaries. Research interfaces enforce chronological validation splits, out-of-sample holdout reporting, and standardized trial records. Python and JSON tool interfaces return structured dictionary outputs, ensuring that all intermediate calculations can be logged and audited.

Local decision models. Adapters for Kev[1] and Laya[2] expose structured decisions and passage reranking from local checkpoints, without hosted API credentials. Real GPU integration checks covered Kev-0.8B, Kev-4B, and the English and multilingual Laya checkpoints, including RAG composition and rejection of oversized inputs. Original responses and checkpoint identities are retained; bounded normalization of rounded probabilities does not establish calibration.

Exploratory routing regression. We froze the same three BM25 skill candidates for each of 108 public regression queries and evaluated a seeded shuffled order and its reversal. BM25 selected the expected skill on 76 queries. Kev-4B selected it on 79 and 82 queries, respectively, but changed its decision on 24 queries when the order changed. Kev-0.8B obtained 45 and 50 correct selections.

[1] `https://github.com/jaredpalmer/kev`
[2] `https://github.com/NandhaKishorM/laya`

Both Laya variants were rejected before inference on all queries because their option budgets would truncate the full skill descriptions; these rejections do not estimate prediction accuracy. The fixtures were previously used in development and contain only two Chinese queries. This comparison measures public routing regression, not unseen financial-task performance or successful tool execution. BM25 remains the default, with Kev-4B available as an optional decision step.

Storage recovery and revision ordering. In a separate engineering study, we terminated collector processes before saving, during a batch, immediately before transaction commit, and after commit returned, with three repeats per boundary. All 12 cases recovered the expected atomic state across event records, latest-version pointers, pending alerts, and the collection cursor; independent read-only database checks confirmed these states. Runs with 100, 1,000, and 10,000 synthetic events retained the expected record counts and cursor. These checks cover process termination, not machine power loss or financial task accuracy. An out-of-order revision test also exposed an interface boundary: the storage layer's `latest` pointer follows arrival order, so an older revision arriving last becomes current. Historical research must therefore apply an explicit chronological or availability-time selection policy rather than infer publication order from that pointer.

Report-QA workflow ablation. On 32 public FinQA questions from 16 companies, we compared BM25S components, the same retrieval with a frozen financial skill, the library RAG interface with that skill, and all supplied report text and tables. Qwen2.5-Coder-14B-Instruct produced 3, 2, 2, and 1 correct answers, respectively; Mistral-Nemo-Instruct-2407 produced none in these conditions. Each model generated 128 answers. Applying the citation-label gate to the same frozen RAG answers changed no outcomes; this paired check adds no independent samples and does not test semantic support. For Qwen, incorrect accepted answers numbered 5, 1, 0, and 5, respectively. The smaller number of wrong acceptances therefore accompanies low correct completion and frequent rejection or format failure, rather than demonstrating an overall accuracy improvement. The full-context condition uses FinQA's extracted text and tables, not complete original annual reports.

Answer model	Reranker	Correct	Errors
Qwen	BGE	2/32	30/32
Qwen	Kev-4B	0/32	31/32
Mistral	BGE	0/32	32/32
Mistral	Kev-4B	0/32	32/32

Table 18: Exploratory FinQA reranking transfer. Errors count saved scoring or answer-format exceptions, not failed compute jobs. One Qwen/Kev answer was incorrect without such an exception. Model identities are given in the text.

Model	Condition	Official	Fence	Recov.	No final
Qwen	RAG	2	2	3	3
Qwen	BGE	2	2	2	3
Qwen	Kev-4B	0	0	2	3
Mistral	RAG	0	0	0	26
Mistral	BGE	0	0	0	24
Mistral	Kev-4B	0	0	0	25

Table 19: Post-hoc FinQA format reanalysis (32 questions per row). Official: original strict program correctness. Fence: single-fence parser. Recov.: recoverable correct final answer, which is not program or citation correctness.

Downstream transfer from passage reranking. Using the same 32 questions, we froze the ten BM25 passage candidates and reranked them with BGE or local Kev-4B before supplying up to five passages within the same context budget. The answer agent, financial skill, calculator, and response limits were held fixed. All 64 planned answers per model were saved. Table 18 reports their original scores, which were independently replayed against the official numerical and program labels. These questions overlap the workflow and memory studies; the 128 generated answers are not 128 independent tasks.

Frozen format reanalysis. To test whether answer formatting alone explains these low scores, we re-scored the 192 saved RAG and reranking answers without regenerating any of them. Target-blind extractions were bound by hash before the official FinQA executor and gold labels were loaded, and the original official counts were reproduced first. Table 19 keeps those program scores, adds the prospective single-fence parser, and separately counts a final answer as recoverable when the parsed program is correct or the final text states one unambiguous correct number. Percent signs are normalized; refusals, ambiguous scales, and missing answers stay failures, and no tool output replaces a missing final answer.

Relaxing the fence adds no correct program. Recoverable final answers exceed program correctness in 3 answers (3 from Qwen), and most Mistral answers never reach a final answer. Formatting is therefore one failure among several, not the explanation for the scores, and the reanalysis does not establish a retrieval or reranking gain.

Interpretation and runtime limitation. The reranking study does not demonstrate a Kev downstream-answer advantage. Answer-format and scoring exceptions dominate these runs. The original Mistral runtime also emitted a tokenizer-regex warning; the frozen run was not modified after inspecting its outcomes. Consequently, its zero correct counts do not isolate model capability from tokenization and answer-format problems. The audit verifies saved outputs, tool receipts, and grades, not the validity of every runtime choice. These public, overlapping tasks do not establish unseen-task generalization, semantic citation support, or human development-time savings.

J Within-task context selection with HiSTrim

Financial agents frequently execute quantitative tools that return verbose tables of historical prices, factor loadings, and optimization logs. Over multi-turn interactions, these intermediate outputs quickly fill the model's context window and slow down generation. Within-task context selection determines which interaction records should remain in active attention and which can be pruned.

HiSTrim prunes intermediate interaction turns while strictly preserving task instructions, problem constraints, and asset identifiers.

J.1 Protected, sign-preserving routing

We partition the context into two sets: protected tokens $\mathcal{P}$ (containing system prompts, task instructions, and the current query) that are never evicted, and historical interaction turns $\mathcal{H}_{\ell-1}$ (such as past tool outputs and intermediate code traces) eligible for pruning at layer ℓ.

For each interaction turn representation h_m, a dual-head router computes a gating logit g_m and a utility logit u_m:

$$g_m = w_g^\top h_m + b_g, \qquad u_m = w_u^\top h_m + b_u, \quad (1)$$

$$r_m = \sigma(g_m) u_m, \qquad \sigma(g) = (1 + e^{-g})^{-1}. \quad (2)$$

Here $\sigma(g_m) \in (0, 1)$ modulates the utility magnitude. Because $\sigma(g_m) > 0$, $\operatorname{sign}(r_m) = \operatorname{sign}(u_m)$. Using the sigmoid gate ensures that positive utility logits always produce positive routing scores, avoiding the sign reversals that occur with activation functions like SiLU when both logits are negative.

Scores are standardized across the sequence and filtered with a learned threshold τ_ℓ:

$$z_m = (r_m - \mu_\ell)/(s_\ell + \epsilon), \quad (3)$$

$$\mathcal{H}_\ell = \{m \in \mathcal{H}_{\ell-1} : z_m \geq \tau_\ell\}, \quad (4)$$

$$\mathcal{T}_\ell = \mathcal{P} \cup \bigcup_{m \in \mathcal{H}_\ell} \mathcal{I}_m, \quad (5)$$

where $\mathcal{I}_m$ denotes the token positions belonging to turn m. This guarantees that protected instruction tokens $\mathcal{P}$ remain in the active set $\mathcal{T}_\ell$ at all layers.

J.2 Forward-only calibration and buffer compaction

Because fine-tuning the full LLM backbone on every task is computationally expensive, the router direction w is calibrated using forward-only finite differences on the unit sphere:

$$w_0 = \frac{\sum_s \alpha_s v_s / \|v_s\|_2}{\|\sum_s \alpha_s v_s / \|v_s\|_2\|_2}. \quad (6)$$

Weights are updated in the tangent space $g_\perp = (I - ww^\top)g$ followed by normalization, allowing the router to adapt without full backpropagation through the LLM.

At inference time, rather than retaining evicted tokens with an attention mask, HiSTrim physically gathers surviving entries into a contiguous packed KV-cache buffer. This reduces active memory bandwidth and accelerates token decoding.

J.3 Attention sink compensation

Transformer attention concentrates heavily on initial prefix tokens (attention sinks; Xiao et al., 2024). Evicting intermediate tokens reduces the softmax denominator, which can distort attention probabilities over surviving tokens. To maintain calibrated attention distributions, we add an offset to the sink logit s_0:

$$\Delta^* = \log\left(1 + \sum_{j \in \mathcal{E}} e^{s_j - s_0}\right), \quad (7)$$

where $\mathcal{E}$ is the set of evicted token indices. In practice, the lightweight count heuristic $\beta \log(|\mathcal{E}| + 1)$ absorbs the removed probability mass without requiring full exponent recomputation.

Empirical performance. On sequential recommendation tasks (Appendix L), HiSTrim achieves up to $2.82\times$ inference speedups while retaining under 50% of historical KV tokens, without degrading generation quality.

K Cross-episode experience and persistent memory

Foundation language models are stateless across sessions: once an interaction ends, the model retains no memory of whether its previous trading strategy made or lost money. Fine-tuning the entire LLM with gradient descent after each trading day is impractical, computationally expensive, and prone to overfitting on noisy financial markets.

To provide cross-episode memory without altering LLM weights, we use an external associative memory module inspired by the fruit-fly mushroom body (Huang et al., 2024). In biological systems, the mushroom body maps complex

sensory features into sparse representations and uses dopamine-gated synaptic plasticity to associate sensory cues with delayed rewards and punishments across multiple decay rates (short- and long-term memory).

Associative memory architecture. Our memory module projects market state features and candidate actions into a high-dimensional sparse representation. When the agent evaluates a new trading decision, it queries the memory module for associative preference scores based on prior experiences in similar market regimes. These scores serve as contextual guidance alongside the prompt, without modifying the frozen LLM backbone.

Updating from complete holding episodes. Financial rewards are inherently delayed: an allocation chosen today only yields returns after a multi-day holding period. The feedback interface models an entry and its subsequent holding days as a single complete decision episode. For positive account values $W_0, \ldots, W_T$, the target return is

$$G = \sum_{t=0}^{T-1} \log(W_{t+1}/W_t) = \log(W_T/W_0). \quad (8)$$

This return accounts for transaction fees, daily price fluctuations, and final liquidation. Memory updates fire only after the position is fully closed and settlement clears.

The verification interface checks timestamp ordering, return accounting, and data lineage. Trades relying on look-ahead data or unadjusted prices are rejected before reaching the memory module, preventing spurious profits from corrupting associative weights. Valid negative returns remain active learning signals, allowing the agent to learn from losses. Each verified trade receipt updates memory exactly once.

Isolating learning from risk filtering. To verify that performance gains reflect genuine associative learning rather than passive risk reduction, our benchmark compares the updating memory against a frozen-memory control and a volatility exposure gate under identical historical splits and costs. This setup isolates experiential adaptation from static volatility filtering.

L Empirical evaluation of selective context (HiSTrim)

This section presents empirical benchmarks evaluating HiSTrim across sequential decision and generative recommendation tasks, assessing its context routing fidelity, KV memory compression, and inference acceleration.

Generative sequential recommendation. Table 20 reports performance across five representative sequential tasks. Across both 1.7B and 8B parameter scales, HiSTrim maintains high retrieval and generation accuracy (e.g., MRR of 19.21 on News with 8B) while reducing the historical KV footprint to under 50%, substantially outperforming static pruning and dense baselines in efficiency.

Calibration-seed stability. Evaluating across five calibration partitions for OneRec-1.7B Ad yields Pass@32 of $18.78\% \pm 0.72\%$ and MRR@32 multiplied by 100 of 3.81 ± 0.19 (mean and sample standard deviation). This confirms that the learned router directions and standardized thresholds remain highly stable across different calibration partitions.

Task accuracy, agreement and throughput. Table 21 presents rating and ranking benchmarks under varying context budgets. On Goodreads with Qwen3-8B, HiSTrim achieves lower MAE (1.078 vs. 1.232) while retaining only 35% of historical KV cache. On the hybrid Amazon ranking task, HiSTrim achieves higher NDCG@10 (0.5567 vs. 0.5183) while providing a $1.78\times$ throughput speedup over the dense model, with the aggressive variant yielding a $2.82\times$ speedup at 35% KV retention.

Model	Method	Hist. KV (%)	Ad	Product	Video	Label-cond.	Interactive
1.7B	Full Dense	100.0	3.24	7.81	5.16	1.25	11.95
1.7B	TokenDrop-50	50.0	3.08	7.72	4.89	1.29	11.70
1.7B	H2O	54.6	3.98	7.51	7.77	1.25	11.72
1.7B	SnapKV	54.6	3.85	7.75	7.36	1.30	11.85
1.7B	Router-Tuning	45.7	4.00	7.27	3.30	1.18	11.24
1.7B	HiSTrim	47.5	4.10	8.09	6.43	1.50	12.08
8B	Full Dense	100.0	4.56	8.22	7.67	1.41	18.25
8B	TokenDrop-50	50.0	4.22	7.89	7.12	1.32	17.40
8B	H2O	54.6	5.06	8.28	8.94	1.37	18.95
8B	SnapKV	54.6	5.02	8.25	8.54	1.72	18.70
8B	Router-Tuning	46.2	4.12	7.95	6.84	1.34	17.65
8B	HiSTrim	49.5	5.25	8.54	7.96	1.47	19.21

Table 20: Generative sequential recommendation performance across tasks: MRR@32 ($\times 100$) evaluated over 1,000 users per task with 32 beams. Historical KV retention excludes protected tokens. HiSTrim matches or exceeds dense retrieval quality while reducing historical KV footprint by more than half.

Task / model	Method	Hist. KV (%)	Label metric	Teacher metric	Speedup
Beauty / Qwen3-0.6B	Dense	100.0	MAE 1.642	KL 0.000	—
	TokenDrop-50	50.0	MAE 1.517	KL 0.233	—
	HiSTrim	50.1	MAE 1.643	KL 0.084	—
Goodreads / Qwen3-8B	Dense	100.0	MAE 1.232	—	—
	HiSTrim	35.0	MAE 1.078	Teacher MAE 0.651	—
Amazon / DCNv2+LM	Dense	100.0	NDCG@10 0.5183	—	$1.00\times$
	TokenDrop-50	50.0	NDCG@10 0.5204	—	$1.79\times$
	HiSTrim	50.0	NDCG@10 0.5567	—	$1.78\times$
	HiSTrim aggressive	35.0	NDCG@10 0.5065	—	$2.82\times$

Table 21: Rating and ranking comparisons across sequential tasks. Lower MAE and KL are better; higher NDCG is better. HiSTrim demonstrates substantial KV compression alongside up to $2.82\times$ wall-clock inference speedups.